Day 2

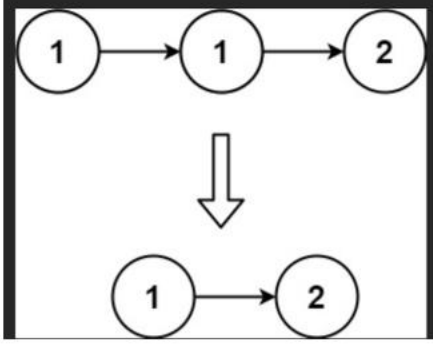
Friday, 14 August 2026 9:27 AM

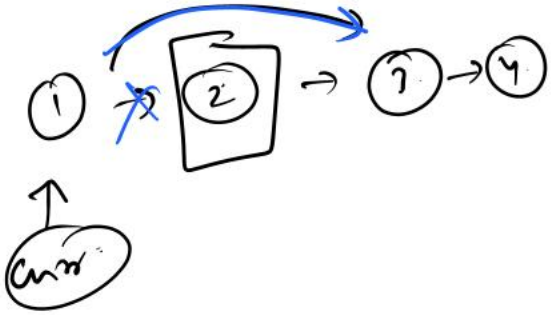
83. Remove Duplicates from Sorted List

Easy | Topics | Companies

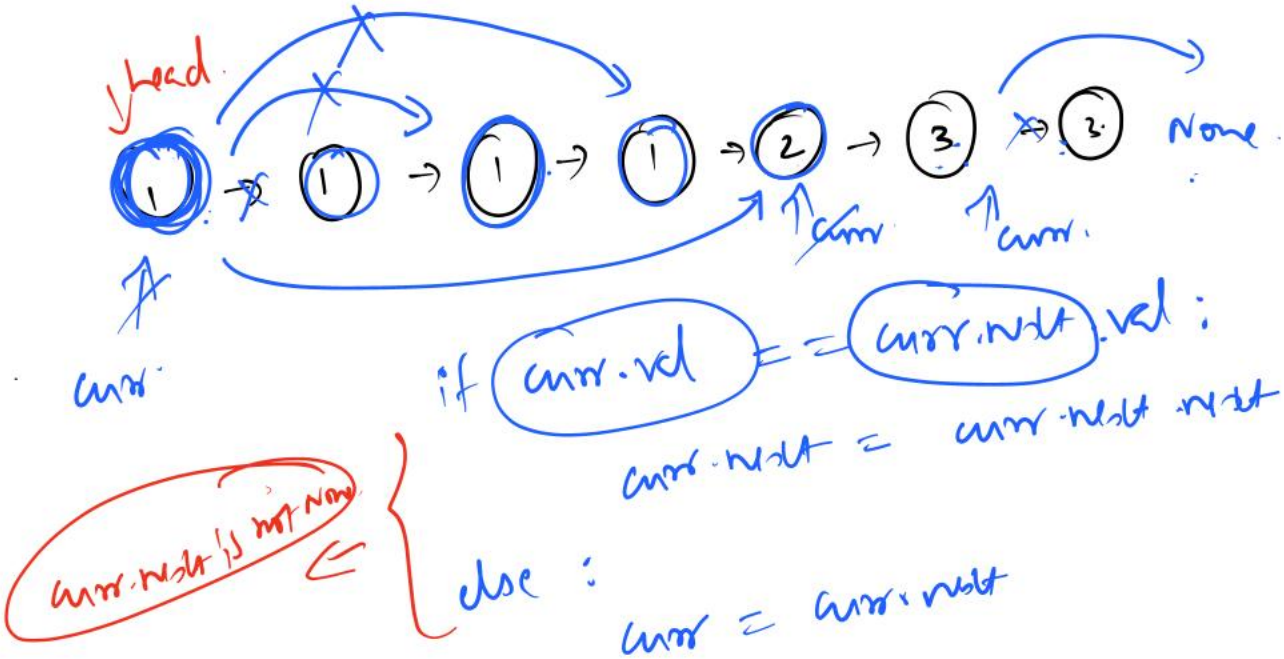
Given the `head` of a sorted linked list, *delete all duplicates such that each element appears only once*. Return *the linked list* ***sorted*** *as well*.

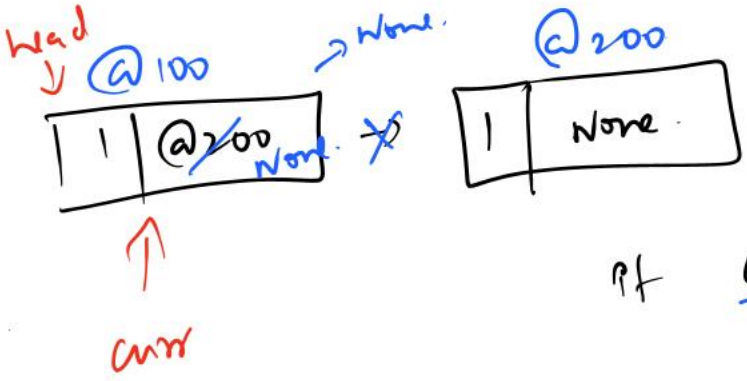
Example 1:





curr.next = curr.next.next





if curr.val == curr.next.val :

curr.next = curr.next.next

curr | 2100

```python
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None :
            return head
        curr = head
        while curr.next :
            if curr.val == curr.next.val :
                curr.next = curr.next.next
            else :
                curr = curr.next
        return head
```

p.no : 83

T.C → O(n)

S.C → O(1)

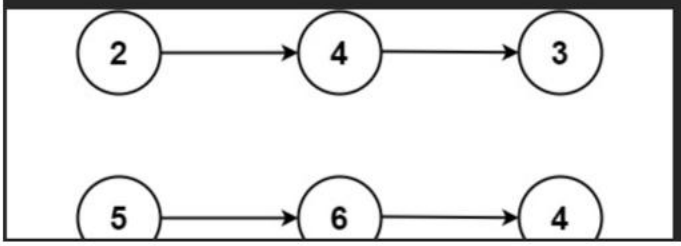
2. Add Two Numbers

Medium | Topics | Companies

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

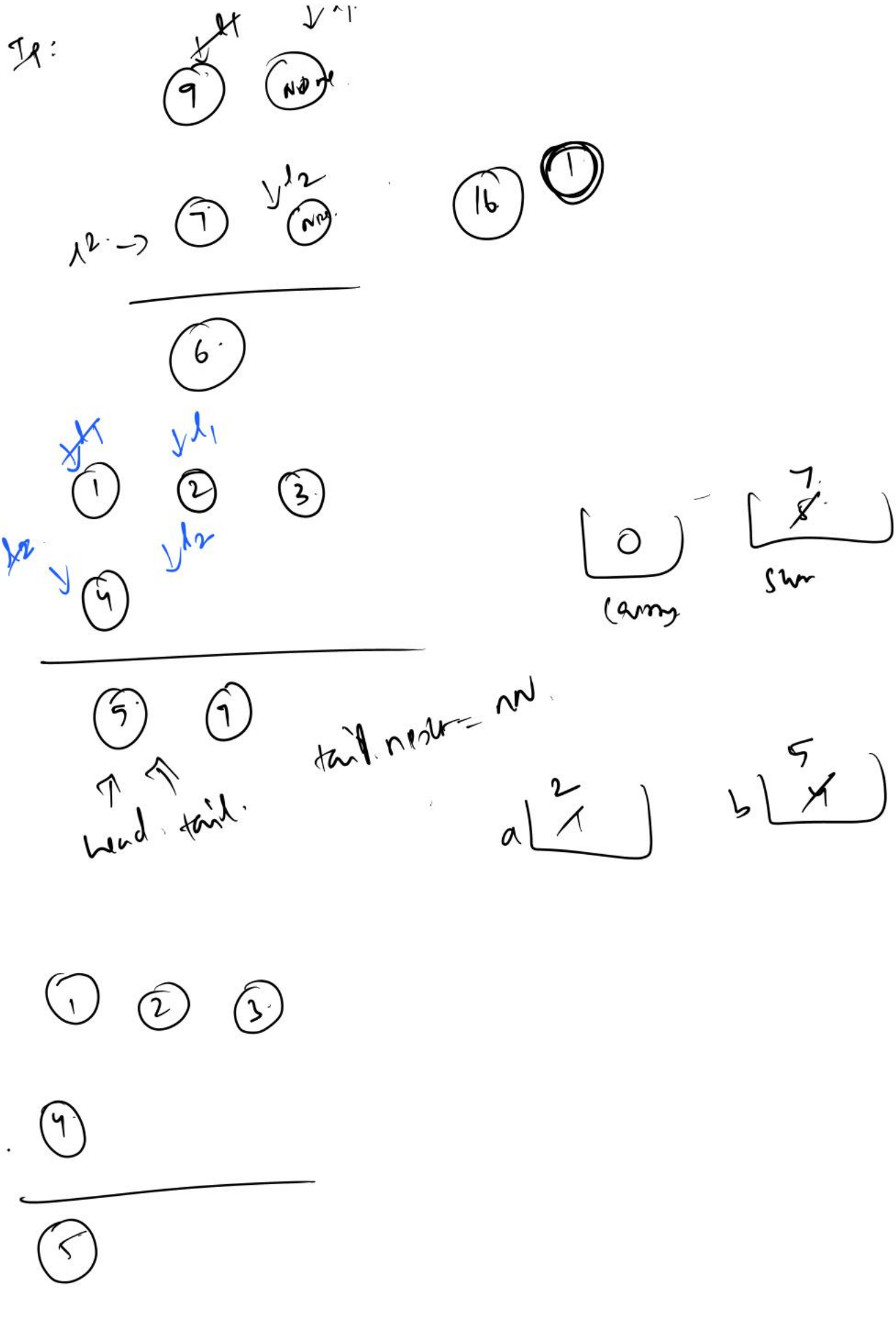
Example 1:



7 → 0 → 8

7 0 8

. 01



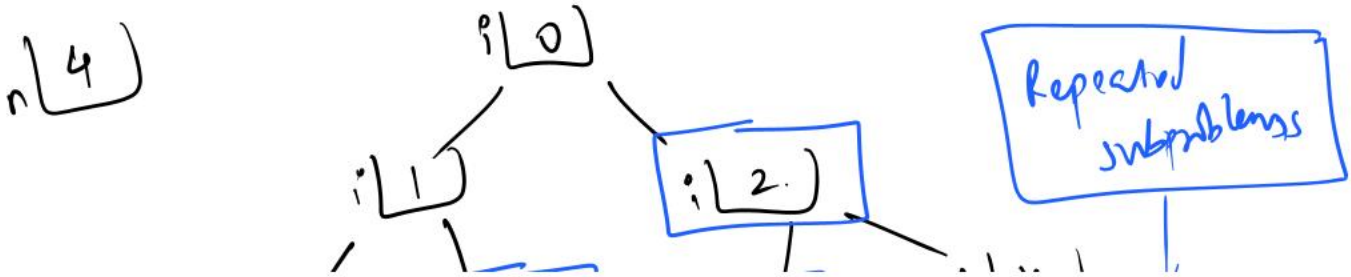
```python
def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]
:
    head = tail = None
    carry = 0
    while l1 or l2 or carry :
        a = l1.val if l1 else 0
        b = l2.val if l2 else 0
        sum = a + b + carry
        newNode = ListNode(sum % 10)
        carry = sum // 10
        if head is None :
            head = newNode
            tail = newNode
        else :
            tail.next = newNode
            tail = newNode
        if l1 :
            l1 = l1.next
        if l2 :
            l2 = l2.next
    return head
```

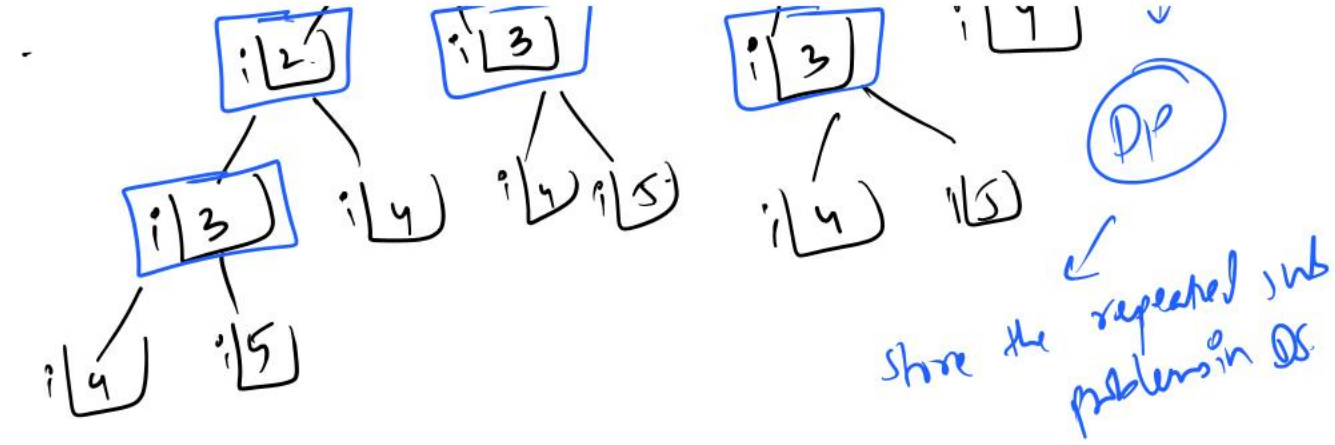
P.No : 2.

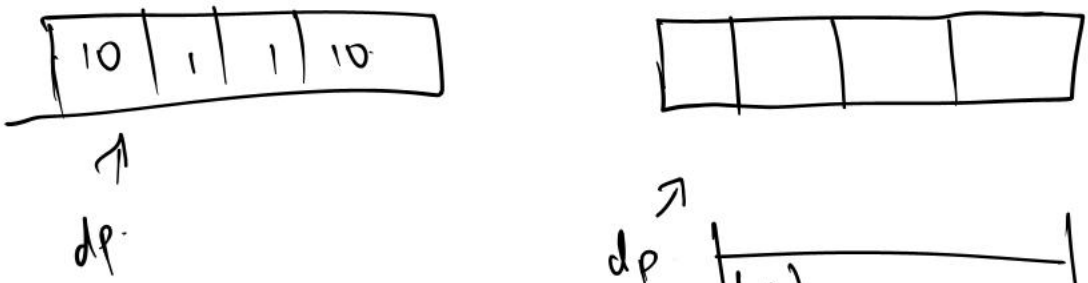
T.C -> O(n)

S.C -> O(n).

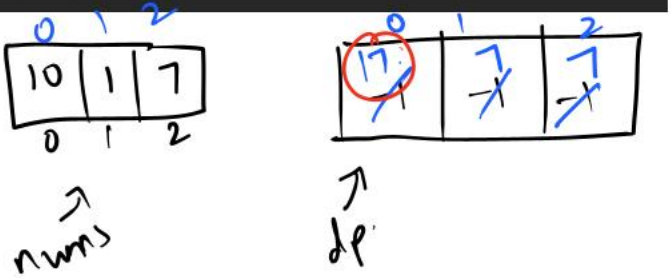
```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i) :
            if i >= len(nums) :
                return 0
            robCurrent = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            return max(robCurrent, skipCurrent)
        return helper(0)
```

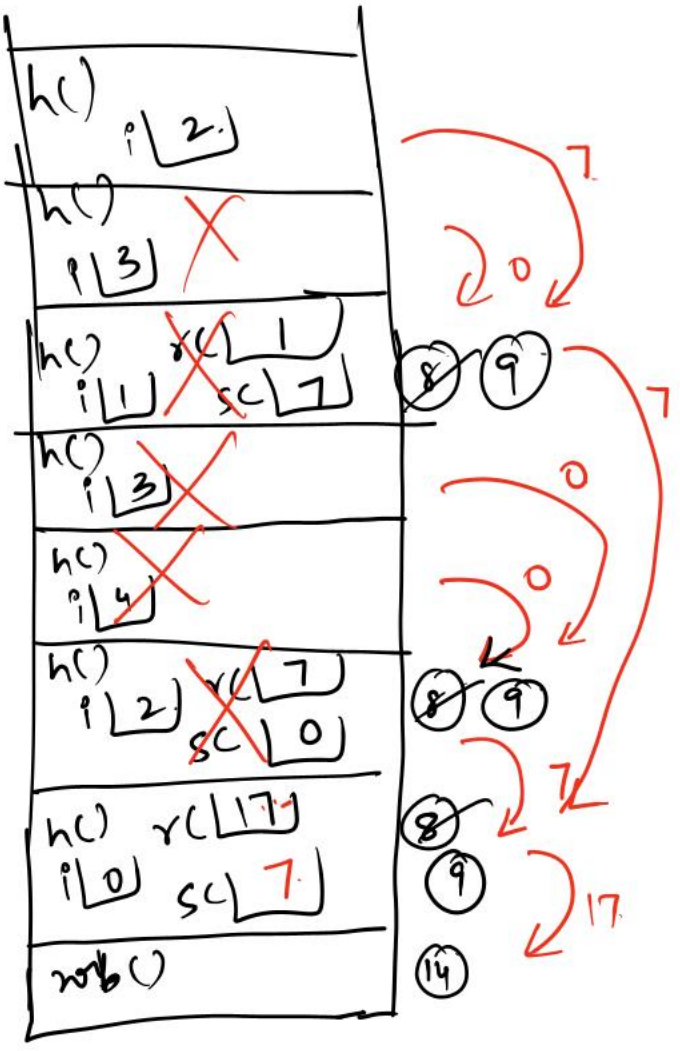







```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i) :
            if i >= len(nums) :
                return 0
            if dp[i] != -1 :
                return dp[i]
            robCurrent = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            dp[i] = max(robCurrent, skipCurrent)
            return dp[i]
        n = len(nums)
        dp = [-1] * n
        return helper(0)
```






```
class Solution:
```

```python
def rob(self, nums: List[int]) -> int:
    def helper(i) :
        if i >= len(nums) :
            return 0
        if dp[i] != -1 :
            return dp[i]
        robCurrent = nums[i] + helper(i + 2)
        skipCurrent = helper(i + 1)
        dp[i] = max(robCurrent, skipCurrent)
        return dp[i]
    n = len(nums)
    dp = [-1] * n
    return helper(0)
```

→ P. No : 198.

→ T.C → O(n).

→ S.C → O(n).

n + n → dp array size

↓

call stack space

T.C → O(n). S.C → O(n) → call stack space + dp array space

↓

n

n → 10

call stack → 10⁴ functions

Recursion error or stack overflow error

10	1	2	5
0	1	2	3

15	6	5	0	5	0	0
0	1	2	3	4	5	

dp

$dp[i] = \max(dp[i+1], nums[i] + dp[i+2])$

-1

6, 10 + 5

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [0] * (n + 2)
        for i in range(n - 1, -1, -1) :
            dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
        return dp[0]
```

T.C → O(n)

S.C → O(n) → dp array size

78. Subsets

Solved

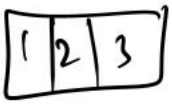
Medium | Topics | Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```



[ ]

[1]

[2]

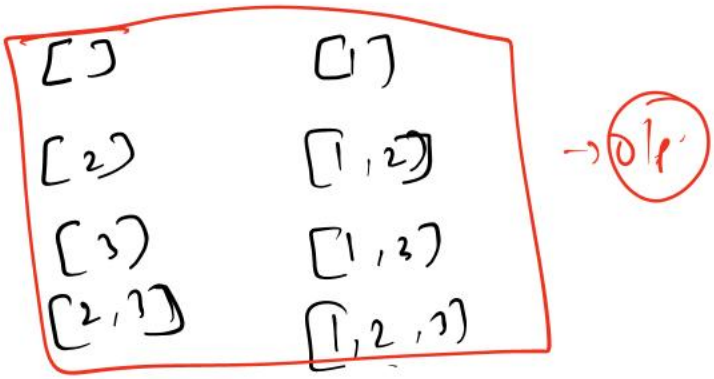
[3]

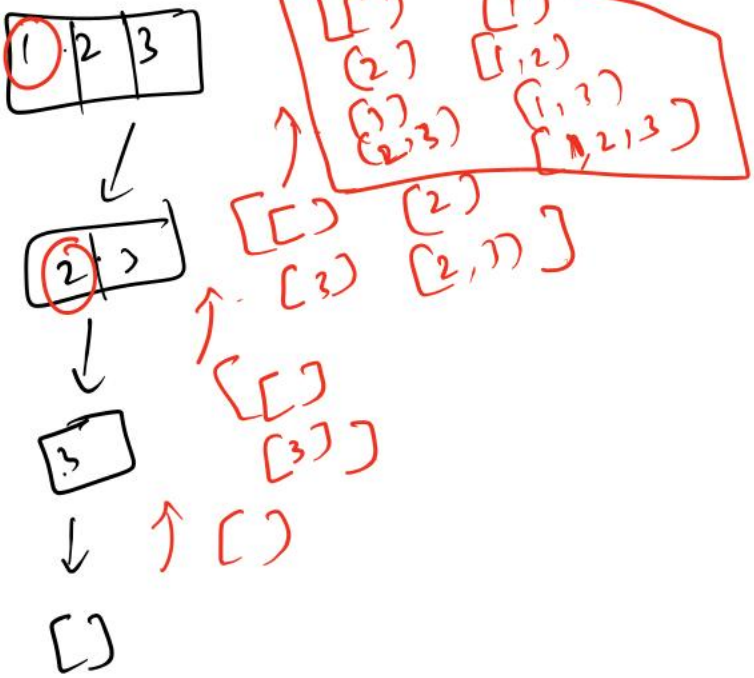
[1, 2]

[1, 3]

[2, 3]

[1, 2, 3]

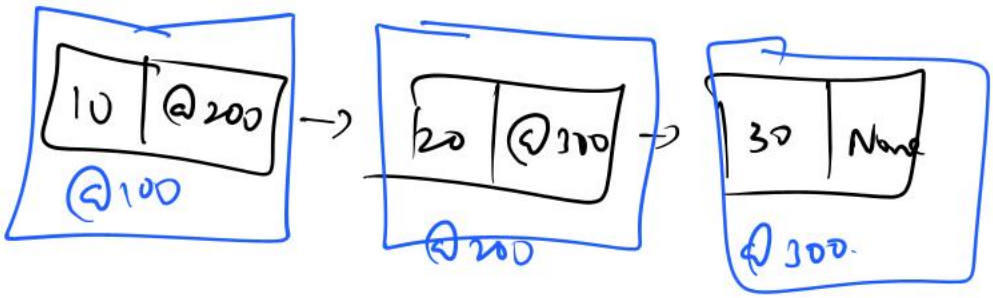





```
def helper(i):
    # if i == n:
        return [[]]
    # so = helper(i+1)
    # output = []
    # copy all the 1D strings
      add(i) th list to so
```

Copy
Return Output

```python
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def helper(i) :
            if i == len(nums) :
                return [[]]
            smallOutput = helper(i + 1)
            output = []
            # 1st half copy from smallOutput
            for row in smallOutput :
                output.append(row)
                output.append([nums[i]] + row)
            return output
        return helper(0)
```



@100

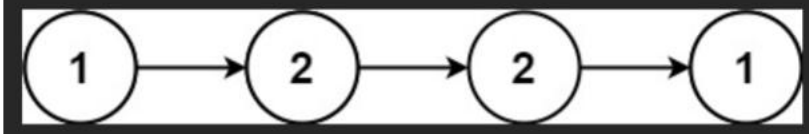
234. Palindrome Linked List

Solved

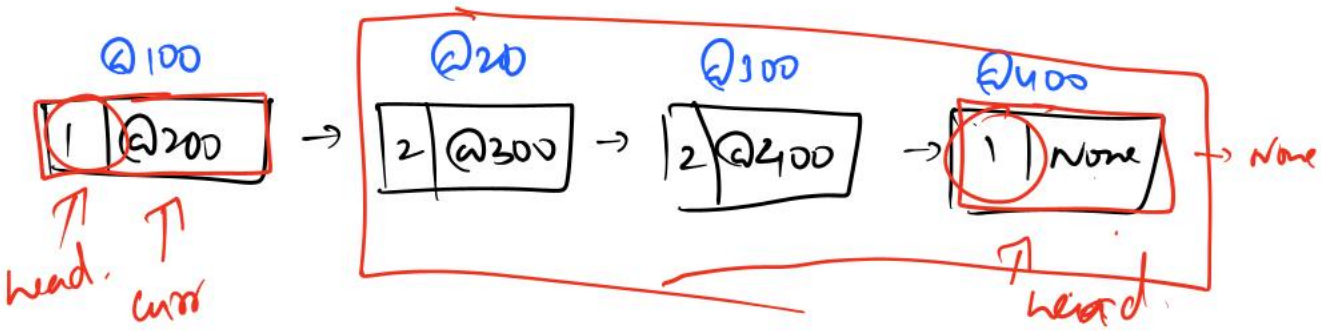
Easy | Topics | Companies

Given the `head` of a singly linked list, return `true` *if it is a palindrome or* `false` *otherwise.*

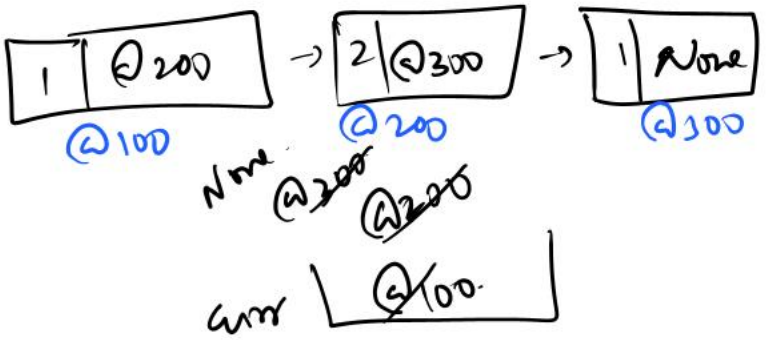
Example 1:

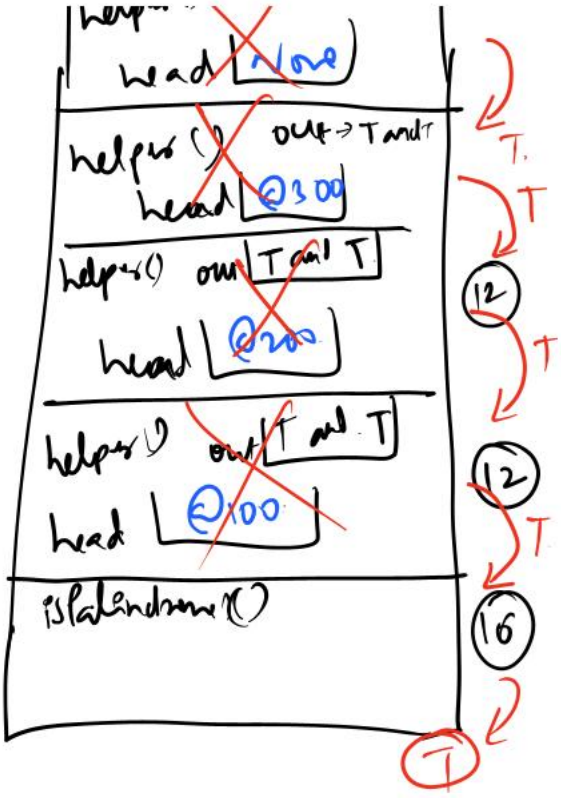



```
Input: head = [1,2,2,1]
Output: true
```



```python
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        def helper(head) :
            nonlocal current
            if head is None :
                return True
            output = helper(head.next) and head.val == current.
            val
            current = current.next
            return output
        current = head
        return helper(head)
```





2130. Maximum Twin Sum of a Linked List

Solved

Medium | Topics | Companies | Hint

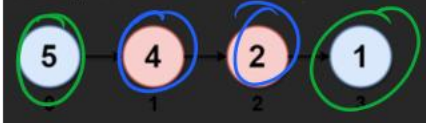
In a linked list of size `n`, where `n` is **even**, the `i^th` node (**0-indexed**) of the linked list is known as the **twin** of the `(n-1-i)^th` node, if `0 <= i <= (n / 2) - 1`.

- For example, if `n = 4`, then node `0` is the twin of node `3`, and node `1` is the twin of node `2`. These are the only nodes with twins for `n = 4`.

The **twin sum** is defined as the sum of a node and its twin.

Given the `head` of a linked list with even length, return *the **maximum twin sum** of the linked list*.

Example 1:




```
Input: head = [5,4,2,1]
```

```python
class Solution:
    def pairSum(self, head: Optional[ListNode]) -> int:
        def helper(head) :
            nonlocal current, maxSum
            if head is None :
                return
            helper(head.next)
            currentSum = head.val + current.val
            maxSum = max(currentSum, maxSum)
            current = current.next

        current = head
        maxSum = 0
        helper(head)
        return maxSum
```